

データ構造 スパーステーブル 区間クエリ モノイド

スパーステーブル

[解説動画はこちら](#)です。

1. 概要

本講座では、静的な配列に対するある種の区間クエリを高速に処理するデータ構造である**スパーステーブル** (Sparse Table) を解説します。

スパーステーブルは、事前計算に $O(N \log N)$ 時間を使うことで、区間最小値などのクエリに $O(1)$ 時間で答える手法です。歴史的には、[文献1](#)における**線形時間 LCA** のアルゴリズムに用いられたことが有名です。

スパーステーブルは、区間長が 2^k のときの情報から区間長が 2^{k+1} のときの情報を構成するというもので、アイデアは二分累乗法 (binary exponentiation) やダブリング (binary lifting) などとも類似しています。ただし、 $\min(x, x) = x$ であることを使ってクエリあたり $O(1)$ 時間を実現する際に、スパーステーブル特有の考え方が現れます。

スパーステーブルは仕組み、実装ともに比較的理解しやすく、累積和の次に学ぶ区間クエリを扱うデータ構造としても適切です。

2. 前提知識

特にありません。

5.1 節ではモノイドや半群に関する用語を使う部分がありますが、スキップ可能です。

3. 問題

3.1. 区間最小値クエリ

本講座では次の問題を通して、スパーステーブルによる区間クエリ処理について説明します。

【問題 1】（区間最小値クエリ）

長さ N の整数列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。 Q 個のクエリに答えてください。 クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので、

$$\min(A_L, A_{L+1}, \dots, A_{R-1})$$

を出力してください。

クエリは区間最小値を求めるもののみで、整数列 A は**変更されることがない**ことに注意してください。 このことを A は**静的** (static) であるといいます。

以下では、半开区間の記号 $[L, R) = \{L, L+1, \dots, R-1\}$ を用います（記号については競技プログラミング基本用語集を参照）。例えば $\min(A_L, A_{L+1}, \dots, A_{R-1})$ を「 $[L, R)$ における A の最小値」などを書くことがあります。

3.2. 素朴な方法

方法 1

クエリが与えられるたびに、そのまま $R - L$ 個の整数の最小値を求めることができます。 クエリごとに $O(N)$ 時間、問題全体では $O(NQ)$ 時間で解くことができます。

方法 2

すべての区間 $[L, R)$ における A の最小値 $\text{ANS}[L][R]$ を求めて配列に格納しておきます。 この際、 L を固定して考えると簡単です。 $\text{ANS}[L][L+1] = A_L$ とした上で、 $i = L+1, L+2, \dots, N-1$ に対して

$$\text{ANS}[L][i+1] = \min(\text{ANS}[L][i], A_i)$$

のように計算していくことができます。 事前計算 $O(N^2)$ 時間、クエリごとに $O(1)$ 時間。 問題全体では $O(N^2 + Q)$ 時間で解くことができます。

4. スパーステーブル

スパーステーブルは、 $O(N \log N)$ 時間の事前計算を行うことで、区間最小値クエリにクエリあたり $O(1)$ 時間で答えるものです。【問題 1】 全体では $O(N \log N + Q)$ 時間で解くことができます。

なお、事前計算で使われるアイデアは、**ダブリング** (binary lifting) として知られる方法と同様です。

4.1. 事前計算

スパーステーブルの事前計算は、**長さが 2 のべき乗であるようなすべての区間 $[L, R)$ における A の最小値を計算し、配列に格納しておく**ものです。(3.2 節の方法 2 のうちごく一部を求めたものと考えることができ、スパーステーブルという名称もこのことからくるものだと考えられます。)

詳細は以下の通りです。まず $2^K \leq N$ を満たす最大の非負整数 K をとります ($N = 0$ の場合の扱いは省略します)。 $k = 0, 1, 2, \dots, K$ および $0 \leq i \leq N - 2^k$ を満たす非負整数の組 (k, i) に対して次の値を計算します。

- $\text{table}[k][i] : [i, i + 2^k)$ における A の最小値。

計算手順は次の通りです。まず $\text{table}[0][i] = A_i$ と初期化します。その後、長さ 2^{k+1} の区間は長さ 2^k の区間 2 つに分割できることから、

$$\text{table}[k+1][i] = \min(\text{table}[k][i], \text{table}[k][i + 2^k])$$

という式を用いて k について昇順にすべての $\text{table}[k][i]$ を求めることができます。

事前計算の計算量は、組 (k, i) の個数と等しく、 $O(N \log N)$ 時間となります。

4.2. クエリ処理

4.1 節の事前計算のもと、区間 $[L, R)$ における A の最小値を以下のように求めることができます。

まず, $k = \lfloor \log_2(R - L) \rfloor$ とします. つまり $2^k \leq R - L < 2^{k+1}$ を満たす非負整数 k をとります. このとき, 長さ 2^k の区間

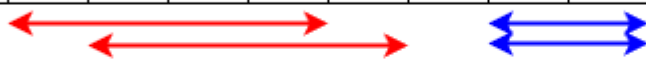
$$[L, L + 2^k), \quad [R - 2^k, R)$$

の和集合がちょうど $[L, R)$ に一致します. このことから, $[L, R)$ における A の最小値は

$$\min(\text{table}[k][L], \text{table}[k][R - 2^k])$$

として求めることができます.

A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	A_8	A_9
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------



$$\min(A_1, A_2, A_3, A_4, A_5) = \min(\text{table}[2][1], \text{table}[2][2])$$

$$\min(A_7, A_8) = \min(\text{table}[1][7], \text{table}[1][7])$$

4.3. $k = \lfloor \log_2 n \rfloor$ について

4.2 節のクエリ処理において, $1 \leq n \leq N$ を満たす整数 n に対して, $k = \lfloor \log_2 n \rfloor$ を求めるという計算が必要になります. このような処理をする方法について述べます.

方法 1 (事前計算)

$1 \leq n \leq N$ に対して $\log_table[n] = \lfloor \log_2 n \rfloor$ を事前計算し, 配列に格納しておきます.

各 k について, $2^k \leq n < 2^{k+1}$ となる n での値を k にしておけばよいです. 事前計算は $O(N)$ 時間, クエリでは配列を参照する $O(1)$ 時間となります.

方法 2 (標準ライブラリや組み込み関数を用いる)

$k = \lfloor \log_2 n \rfloor$ は, $n = \sum_i a_i 2^i$ と 2 進法表記したときに, $a_i = 1$ であるような i の最大値です. 2^k は n の **最上位セットビット** (most significant set bit), k は **最上位セットビットインデックス** と呼ばれます.

▶ 用語について

多くの言語・処理系では、32 bit や 64 bit の正整数に対する最上位セットビットインデックスを求めるための関数が用意されています。例えば $0 < n < 2^{32}$ としたとき

- C++
 - ``31 - __builtin_clz(n)`` (GCC 拡張)
 - ``std::bit_width(n) - 1`` (C++20 以上)
- Python :
 - ``n.bit_length() - 1``

などを利用することが考えられます。使用する言語やバージョンに応じて各自ご確認ください。

以上をまとめると、【問題 1】はスパーステーブルを用いて $O(N \log N + Q)$ 時間で解けることが分かりました。なお、空間計算量は $O(N \log N)$ です。

4.4. 実装例

Library Checker [Static RMQ](#) への提出です。

- <https://judge.yosupo.jp/submission/349017> (4.3 節方法 1)
- <https://judge.yosupo.jp/submission/349019> (4.3 節方法 2)

5. 一般化

スパーステーブルをモノイドの区間積クエリについて適用する場合、どのようなモノイドについて一般化できるかを確認します。

モノイドの用語に馴染みがない場合は、5.1 節は飛ばして（あるいはある程度雰囲気を感じて）、5.2 節の具体例について区間クエリが行えることを理解するようにしてください。または、[代数構造用語集](#) を参照してください。

なお、（定義が必要な用語が増えることを避けるために）モノイドについて説明していますが、モノイドの部分半群に置き換えてもほとんど同じ議論が成立します。

5.1. モノイドに対する区間積クエリ

【問題 2】

$(M, *)$ をモノイドとします. M の元からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます. Q 個のクエリに答えてください. クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので,

$$A_L * A_{L+1} * \dots * A_{R-1}$$

を出力してください.

この問題がスパーステーブルで解けるためには, どのような条件が必要でしょうか?

重要なのは 4.2 節の内容で, $[L, p)$ と $[q, R)$ の和集合が $[L, R)$ に等しいならば,

$$(A_L * \dots * A_{R-1}) = (A_L * \dots * A_{p-1}) * (A_q * \dots * A_{R-1})$$

が成り立つという条件が必要です.

- $L \leq i < q$ に対する総積を x
- $q \leq i < p$ に対する総積を y
- $p \leq i < R$ に対する総積を z

とすれば, これは $x * y * z = (x * y) * (y * z)$ と言い換えられます (ただし, 空積は単位元であるとして). ここから, $y = y * y$ という条件が成り立てばスパーステーブルで **【問題 2】** が解けることが分かります.

定義, 定理としてまとめておきます.

【定義 3】(べき等モノイド)

$(M, *)$ をモノイドとする. $x \in M$ に対して $x * x = x$ が成り立つとき, x は**べき等** (idempotent) であるという.

任意の $x \in M$ がべき等であるとき, M は**べき等** (idempotent) であるという.

【定理 4】

$(M, *)$ がべき等モノイドであるとき, 【問題 2】 をスパーステーブルを用いて解くことができる.

M の元に対する操作 (配列への読み書きやモノイド演算) が $O(1)$ 時間であるとした場合, 計算量は $O(N \log N + Q)$ 時間である.

5.2. 具体例

べき等なモノイド $(M, *)$ の代表例は次の通りです.

M	$*$
$\mathbb{Z} \cup \{\infty\}$	min
$\mathbb{Z} \cup \{-\infty\}$	max
非負整数全体	gcd
$2^w - 1$ 以下の非負整数全体	AND
$2^w - 1$ 以下の非負整数全体	OR

ここでは単位元を持たせるために, min, max に関するモノイドを $\mathbb{Z} \cup \{\pm\infty\}$ としましたが, 適当な区間 $[L, R)$ としてもよいです. その場合 min の単位元は $R - 1$, max の単位元は L となります. また, スパーステーブルは演算の単位元を意識せずとも自然に実装できるため, 単に整数の min, max と理解しても問題ありません.

6. 関連問題

1. <https://judge.yosupo.jp/problem/staticrmq>
2. https://atcoder.jp/contests/abc282/tasks/abc282_f
3. https://atcoder.jp/contests/code-festival-2014-qualb/tasks/code_festival_qualB_d
4. <https://yukicoder.me/problems/no/1036>

5. <https://codeforces.com/problemset/problem/689/D>
6. <https://codeforces.com/problemset/problem/514/D>
7. <https://codeforces.com/contest/1547/problem/F>

▶ 問題について

7. まとめ

本講座では、静的な列に対する区間最小値クエリなどを高速に処理するためのデータ構造であるスパーステーブルを解説しました。

最後に簡単に、他の手法との関連について触れておきます。

まず **【問題 1】** を解く手法としては、競技プログラミングで最も代表的なのは**セグメント木**です。これは **【問題 1】** を $O(N + Q \log N)$ 時間で解くものです。 $N = Q$ 程度の状況ではスパーステーブルとセグメント木は同程度の計算量ということになります。ただしセグメント木の方が空間計算量が小さく、また要素の変更に対応できるため、 $N = Q$ 程度の状況だとセグメント木を用いることの方が多いと思います。

一方で、 Q が N に比べてある程度大きいときには、スパーステーブルによる解法がセグメント木による解法よりも高速になることが多いです。例えば $Q = N \log N$ のときにはスパーステーブルの解法は $O(N \log N)$ 時間、セグメント木の解法は $O(N \log^2 N)$ 時間となります。二分探索内で区間クエリを処理する解法などがこのタイプになることがあります。

また、同じ計算量 $O(N \log N + Q)$ を実現するものとして、**DST** があります (**Disjoint Sparse Table**)。DST は **【問題 2】** を、モノイドがべき等という制約なしに解けるデータ構造なので、ある意味ではスパーステーブルの上位互換です。

【問題 1】 をクエリあたり $O(1)$ 時間で解く方法としては、他にも **Sqrt Tree** などのデータ構造が知られています (**Sqrt Tree**)。Sqrt Tree は **【問題 2】** を $O(N \log \log N + Q)$ 時間で解くものです。また、**線形時間 RMQ** という **【問題 1】** を $O(N + Q)$ 時間で解く方法もあります。

このように、計算量オーダーの意味で **【問題 1】** をより高速に解く代替手法も多くあります。ただし同じ「クエリあたり $O(1)$ 」といってもそこで必要とする配列アクセスや $\min$ 演算の回数などに違いがあり、特に Q が非常に大きい場合にはスパーステーブルがこれらの手法

よりも良いパフォーマンスを達成することも珍しくありません。（同じように、 N がとても小さく Q がとても大きい状況では 3.2 節方法 2 が最も優れた方法となることもあります。）

8. 参考文献

1. Michael A. Bender, Martín Farach-Colton. The LCA Problem Revisited. Proceedings of LATIN 2000: Theoretical Informatics (LATIN 2000). Lecture Notes in Computer Science. Vol. 1776. pp. 88–94. Springer. 2000. DOI: 10.1007/10719839_9. https://doi.org/10.1007/10719839_9.
2. cp-algorithms. Sparse Table. https://cp-algorithms.com/data_structures/sparse-table.html. (閲覧日: 2026-03-02).
3. Scrapbox. data-structures: Sparse Table. https://scrapbox.io/data-structures/Sparse_Table. (閲覧日: 2026-03-02).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)